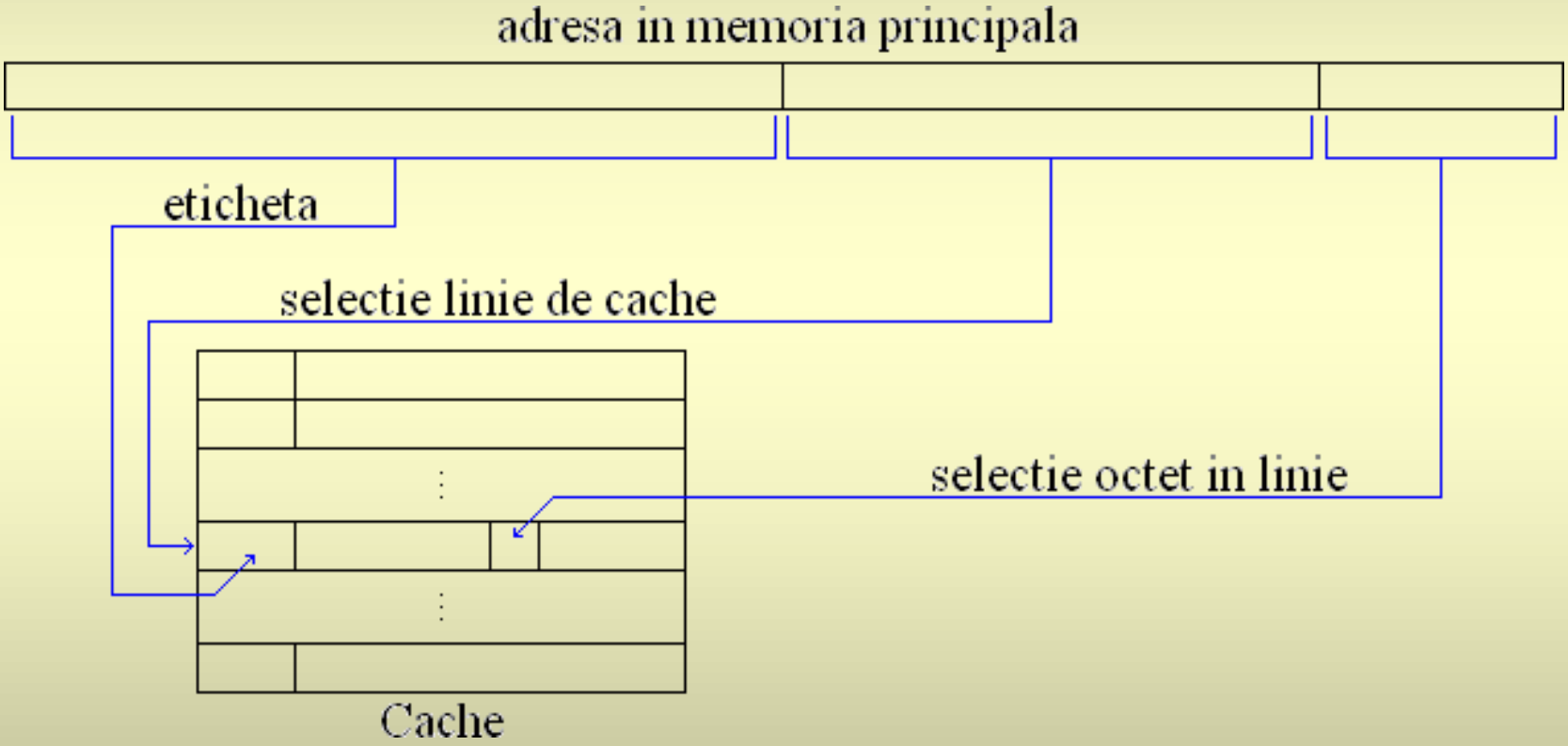
Cache cu adresare directă (1)

- plasarea unei locaţii în cache
  - linia de cache este întotdeauna aceeaşi
  - depinde de adresa din memoria principală
- adresa din memoria principală - 3 părţi
  - eticheta - se memorează în cache
  - selectorul liniei de cache
  - selectorul octetului în cadrul liniei

Cache cu adresare directă (2)



Cache cu adresare directă (3)

Exemplu

- adresa în memoria principală - 32 biţi
- dimensiune cache: 2^{11} linii × 2^5 octeţi/linie
- adresa în memoria principală se împarte în
  - selectorul liniei de cache - 11 biţi
  - selectorul octetului în cadrul liniei - 5 biţi
  - eticheta - 16 biţi (= 32 – 11 – 5)

Cache cu adresare directă (4)

Exemplu (continuare)

- adresa în memoria principală: $45097373_{(10)}$

$0000001010110000001000011001 1101_{(2)}$

- eticheta: $0000001010110000_{(2)}=688_{(10)}$
- linia de cache: $00100001100_{(2)}=268_{(10)}$
- octetul în cadrul liniei: $11101_{(2)}=29_{(10)}$

Cache cu adresare directă (5)

Exemplu (continuare)

- Ce adrese din memoria principală sunt aduse în linia de cache?

$0000001010110000001000011 00xxxxx_{(2)}$

$0000001010110000001000011 00000000 \div$

$0000001010110000001000011 00011111_{(2)}$

$45097344 \div 45097375_{(10)}$

Cache cu adresare directă (6)

Conţinutul unei linii de cache

- un bit care indică dacă linia conţine date valide
  - iniţial, toate liniile sunt goale, deci invalide
- câmpul etichetă
- datele propriu-zise, aduse din memoria principală

Cache cu adresare directă (7)

Avantaje

- implementare simplă
- timp de acces (T_c) redus

Dezavantaje

- lipsă de flexibilitate
- politică de înlocuire neperformantă - rată de succes (H) scăzută

Cache cu adresare directă (8)

Exemplu

```
for(i=0;i<1000;i++) a=a+i;
```

- adrese: `i`→3806240, `a` →1756566572
- ambele sunt memorate în cache în linia 161
- accese alternative → înlocuiri dese în cache → număr mare de ratări

Cache total asociativ (1)

- realizat cu memorii asociative
  - memoria obişnuită - acces la o locaţie pe baza adresei sale
  - memoria asociativă - permite şi regăsirea locaţiei pe baza conţinutului său
  - implementare - valoarea căutată este comparată în paralel cu toate locaţiile
    - de ce în paralel?

Cache total asociativ (2)

Avantaje

- plasarea datelor din memoria principală - în orice linie de cache
- se pot alege convenabil adresele aduse în linia de cache
- se pot implementa politici de înlocuire eficiente - rată de succes (H) ridicată

Cache total asociativ (3)

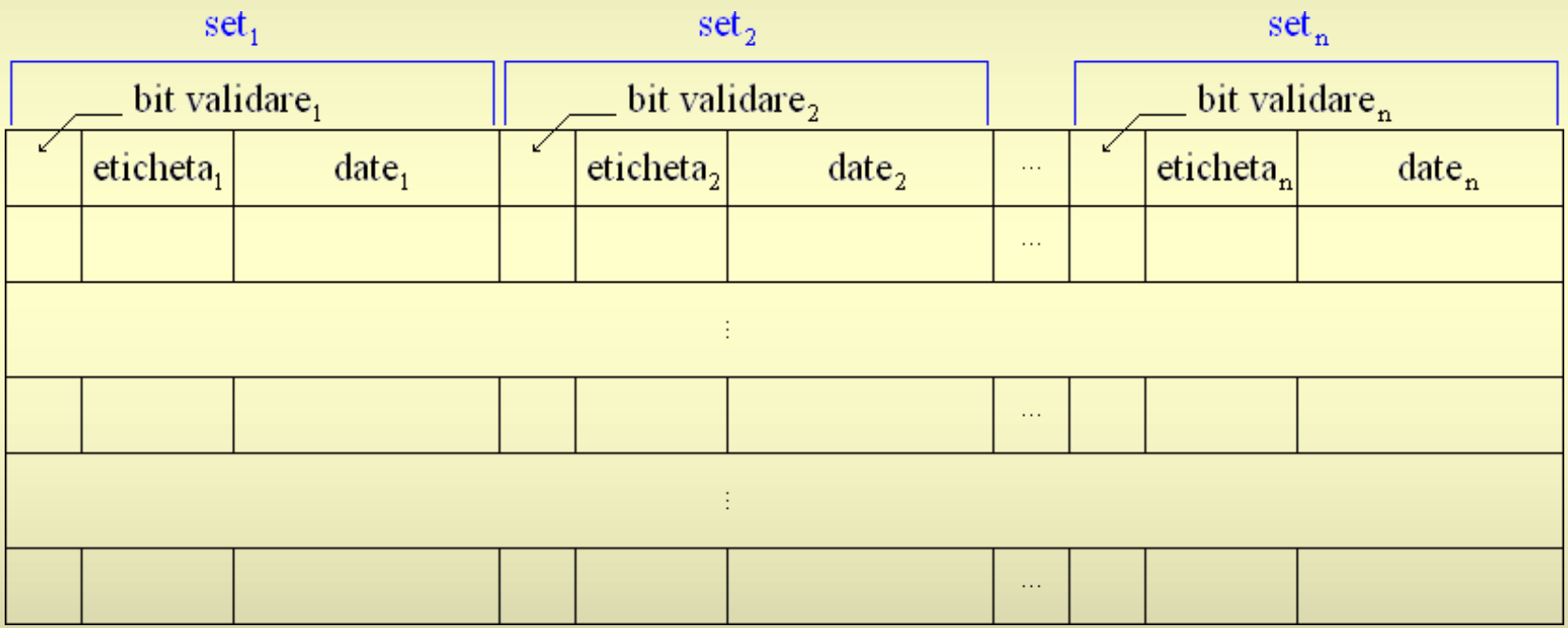
Dezavantaje

- timp de acces (T_c) mare
  - memoriile asociative - lente
  - algoritmi complecşi de înlocuire - timp suplimentar consumat
- hardware complicat pentru memoriile asociative şi algoritmii de înlocuire

Cache parţial asociativ (1)

- numit şi cache asociativ pe seturi
- derivat din cache-ul cu adresare directă
- fiecare linie de cache conţine mai multe seturi de date (4, 8, 16, ...)
- structura unui set
  - bit de validare
  - etichetă
  - date din memoria principală

Cache parţial asociativ (2)



Cache parţial asociativ (3)

Timpul de acces (T_c)

- puţin mai mare decât la cache-ul cu adresare directă
  - trebuie verificate toate cele *n* seturi

Rata de succes (H)

- ridicată
  - elimină problema suprapunerilor

Scrierea în cache (1)

- scriere într-o locaţie care se află în cache
- unde se face scrierea?
- variante
  - doar în memoria principală - nu se poate
    - de ce?
  - doar în cache (*write-back*)
  - atât în cache, cât şi în memoria principală (*write-through*)

Scrierea în cache (2)

Cache de tip *write-back*

- scrierea se face doar în cache
- datele ajung în memoria principală doar la evacuarea din cache
- viteză mare
- probleme în sistemele multiprocesor

Scrierea în cache (3)

Cache de tip *write-through*

- scrierea se face atât în cache, cât şi în memoria principală
- mai lent
  - datorită accesului la memoria principală
- ambele tipuri de cache sunt larg folosite

Scrierea în cache (4)

- dar dacă locaţia modificată nu se află în cache?
- variante
  - locaţia este adusă în cache (*write allocate*)
  - locaţia nu este adusă în cache, ci valoarea sa este actualizată în memoria principală (*no-write allocate*)

Memoria cache în sisteme paralele

- sistem de calcul paralel - mai multe procesoare
  - sau nuclee
- procesoarele pot accesa o zonă de memorie comună
  - pentru a conlucra în rezolvarea problemelor
  - ex.: variabile comune (partajate)
  - dar fiecare procesor are propriul cache

Coerenţa memoriei

- trebuie ca toate procesoarele să folosească ultima valoare scrisă pentru o variabilă partajată
- problema - memoriile cache
- scopul - orice variabilă partajată să aibă aceeaşi valoare
  - în toate cache-urile
  - în memoria principală

Coerenţa - cache *write-back*

`x` - variabilă partajată

Procesor	Acţiune	Valoare cache A	Valoare cache B	Valoare memorie
				9
A	`i=x;`	9		9
B	`j=x;`	9	9	9
A	`x=5;`	5	9	9
B	`k=x;`	5	9	9

Coerenţa - cache *write-through*

`x` - variabilă partajată

Procesor	Acţiune	Valoare cache A	Valoare cache B	Valoare memorie
				9
A	`i=x;`	9		9
B	`j=x;`	9	9	9
A	`x=5;`	5	9	5
B	`k=x;`	5	9	5

Ce înseamnă coerenţa? (1)

1. ordinea execuţiei
   a) procesorul P scrie în variabila X
   b) apoi procesorul P citeşte X
   – între a) şi b) nu există alte scrieri în X

→ citirea b) returnează valoarea scrisă de a)

Ce înseamnă coerenţa? (2)

2. viziune coerentă a memoriei
   a) procesorul P scrie în variabila X
   b) apoi procesorul Q (Q≠P) citeşte X
   – între a) şi b) nu există alte scrieri in X
   – între a) şi b) a trecut suficient timp

→ citirea b) returnează valoarea scrisă de a)

Ce înseamnă coerenţa? (3)

3. serializarea scrierilor
   a) procesorul P scrie în variabila X
   b) procesorul Q (Q=P sau Q≠P) scrie în variabila X

→ toate procesoarele văd cele două scrieri în aceeaşi ordine

– nu neapărat a) înaintea b)

Menţinerea coerenţei cache-urilor

- protocoale de menţinere a coerenţei
- se bazează pe informaţiile despre liniile de cache
  - *invalid* - datele nu sunt valide
  - *dirty* - doar cache-ul curent deţine valoarea actualizată
  - *shared* - cache-ul curent deţine valoarea actualizată, la fel memoria principală şi eventual alte cache-uri

Tipuri de protocoale

- *directory based*
  - informaţiile despre fiecare linie de cache - ţinute într-un singur loc
- *snooping*
  - fiecare cache are o copie a liniei partajate
  - fără centralizarea informaţiei
  - cache-urile monitorizează magistrala
    - detectează schimbările produse în liniile de cache

Actualizarea cache-urilor

- fiecare cache anunţă modificările făcute
- celelalte cache-uri reacţionează
- contează doar operaţiile de scriere
- variante
  - scriere cu invalidare (*write invalidate*)
  - scriere cu actualizare (*write update*)

Scriere cu invalidare (1)

- un procesor modifică o dată
- modificarea se face în cache-ul propriu
  - toate celelalte cache-uri sunt notificate
- celelalte cache-uri
  - nu au o copie a datei modificate - nici o acţiune
  - au o copie a datei modificate - îşi invalidează linia corespunzătoare
  - valoarea corectă va fi preluată când va fi nevoie

Scriere cu invalidare (2)

`x` - variabilă partajată

Proc.	Acţiune	Reacţie cache	Cache A	Cache B	Memorie
					9
A	`i=x;`	read miss	9		9
B	`j=x;`	read miss	9	9	9
A	`x=5;`	invalidation	5	inv.	9
B	`k=x;`	read miss	5	5	5

Scriere cu actualizare (1)

- un procesor modifică o dată
- modificarea se face în cache-ul propriu
  - toate celelalte cache-uri sunt notificate
  - se transmite noua valoare
- celelalte cache-uri
  - nu au o copie a datei modificate - nici o acţiune
  - au o copie a datei modificate - preiau noua valoare

Scriere cu actualizare (2)

`x` - variabilă partajată

Proc.	Acţiune	Reacţie cache	Cache A	Cache B	Memorie
					9
A	`i=x;`	read miss	9		9
B	`j=x;`	read miss	9	9	9
A	`x=5;`	invalidation	5	5	5
B	`k=x;`	read hit	5	5	5

Invalidare vs. actualizare (1)

- mai multe scrieri succesive în aceeaşi locaţie
  - *write invalidation* - o singură invalidare (prima dată)
  - *write update* - câte o actualizare pentru fiecare scriere
  - mai avantajos - invalidare

Invalidare vs. actualizare (2)

- mai multe scrieri în aceeaşi linie de cache
  - modificarea unei locaţii necesită invalidarea/actualizarea întregii linii
  - *write invalidation* - o singură invalidare (prima dată)
  - *write update* - câte o actualizare pentru fiecare scriere
  - mai avantajos - invalidare

Invalidare vs. actualizare (3)

- "timpul de răspuns"
  - timpul între modificarea unei valori la un procesor şi citirea noii valori la alt procesor
  - *write invalidation* - întâi invalidare, apoi citire (când este necesar)
  - *write update* - actualizare imediată
  - mai avantajos - actualizare

Invalidare vs. actualizare (4)

- ambele variante au avantaje şi dezavantaje
- scrierea cu invalidare - ocupare (mult) mai redusă a memoriei şi magistralelor
- scrierea cu actualizare - rata de succes a cache-urilor mai mare
- mai des folosită - invalidarea

Conceptul de cache - extindere

- aplicabil nu doar la procesoare
- tipul de problemă: comunicarea cu o entitate lentă, de mare capacitate
- soluţia: interpunerea unei entităţi cu capacitate mai mică şi viteză mai mare
  - reţine ultimele date vehiculate

Unde mai putem folosi ideea?

Aplicabilitate

- oriunde funcţionează legile localizării
- hardware
- software

Exemple (1)

Cache-uri de disc

- 2 direcţii de aplicare
  - hardware - circuit de memorie integrat în controller
  - software - o zonă din memoria sistem
- entitatea mai mare şi lentă - discul
- entitatea mai mică şi rapidă - memoria

Exemple (2)

Browserul web

- ultimele pagini accesate sunt reţinute pe disc
  - numai localizare temporală - de ce?
- entitatea mai mare şi lentă - reţeaua (Internet)
- entitatea mai mică şi rapidă - discul